Desarrollo de un sistema RPC sobre una instancia EC2

Programación distribuida en Python

De la comunicación por mensajes a la llamada a procedimientos remotos

1. Objetivo

El objetivo de esta práctica es desarrollar un sistema cliente-servidor en Python que permita ejecutar procedimientos de forma remota mediante una capa de RPC (Remote Procedure Call).

El cliente se ejecutará en tu ordenador y el servidor se desplegará en una instancia Amazon EC2.

Esquema



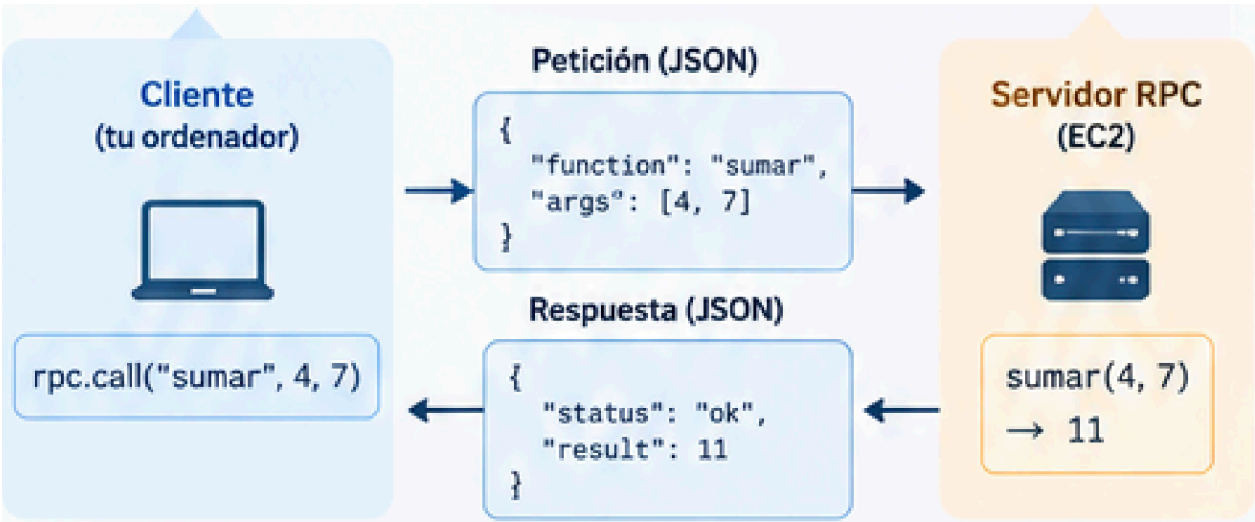
Una llamada en tu código... → ...una ejecución en la nube.

2. Qué vas a aprender

- Comunicación entre procesos mediante sockets TCP.
- Diseño e implementación de un protocolo de mensajes.
- Construcción de una capa RPC que oculte la comunicación.
- Ejecución de procedimientos remotos en una instancia EC2.
- Gestión de errores, timeouts y situaciones de fallo.
- Servidores concurrentes que atienden múltiples clientes.
- Despliegue y configuración básica en AWS.

3. Funcionamiento general

El cliente invoca un procedimiento remoto mediante una llamada a una función de la clase `RPCClient`. La petición se serializa en formato JSON, se envía por TCP al servidor en EC2, se ejecuta el procedimiento solicitado y se devuelve el resultado.



4. Fases de la práctica

1. **Despliegue en EC2:** Crear y configurar una instancia EC2, abrir el puerto necesario, conectarse por SSH y desplegar el servidor.
2. **Comunicación mediante mensajes:**
   - Usar sockets TCP y un protocolo sencillo basado en JSON.
   - Comprobar los mensajes enviados y recibidos.
3. **Construcción de la capa RPC:** Implementar la clase `RPCClient` que permita realizar llamadas remotas de forma transparente.
4. **Servidor RPC:** Registrar y ejecutar distintos procedimientos remotos.
5. **Gestión de errores:** Manejar procedimientos inexistentes, parámetros incorrectos, excepciones durante la ejecución y timeouts.
6. **Concurrencia:** Modificar el servidor para que pueda atender varios clientes simultáneamente (usando hilos o un pool de hilos).

5. Ejemplo de uso

Una vez implementada la capa RPC, desde el cliente podrás invocar procedimientos remotos de forma muy sencilla:

```
from rpc_client import RPCClient

rpc = RPCClient("<IP_EC2>", 5000)

print(rpc.call("sumar", 4, 7))          # 11
print(rpc.call("multiplicar", 3, 5))    # 15
print(rpc.call("tarea_lenta", 2))       # 2 (después de 2 segundos)
```

6. Procedimientos remotos de ejemplo

El servidor incluirá algunos procedimientos de prueba, como:

```
def sumar(a, b):
    return a + b

def multiplicar(a, b):
    return a * b

def potencia(a, b):
    return a ** b

def tarea_lenta(segundos):
    import time
    time.sleep(segundos)
    return segundos
```

> Deberás implementar el mecanismo genérico de registro y ejecución de procedimientos, y añadir al menos dos procedimientos adicionales diseñados por ti.
>
> 💡 Se valorará la originalidad y utilidad de los procedimientos propuestos.

7. Qué se proporciona

```
practica_rpc/
│
├── client/
│   ├── client_example.py     # Ejemplo de uso
│   └── rpc_client.py         # Esqueleto a completar
│
├── server/
│   ├── server.py             # Servidor básico
│   ├── procedures.py         # Funciones de ejemplo
│   └── rpc_server.py         # Esqueleto a completar
│
├── common/
│   └── protocol.py           # Utilidades de protocolo
│
├── tests/
│   └── test_rpc.py           # Pruebas básicas
│
└── README.md                 # Instrucciones iniciales
```

Incluye

- ✅ Estructura del proyecto
- ✅ Código básico de conexión (con envío/recepción simple)
- ✅ Funciones de ejemplo
- ✅ Instrucciones para crear y configurar la instancia EC2
- ✅ Pruebas básicas

No se proporciona

- ❌ Implementación de `RPCClient`
- ❌ Dispatcher del servidor
- ❌ Gestión de errores y timeouts
- ❌ Concurrencia
- ❌ Procedimientos adicionales

8. Entrega

Código fuente

Completo y documentado.

Vídeo corto

Máx. 5 minutos, mostrando el funcionamiento del sistema.

Memoria

2-3 páginas explicando la arquitectura, el protocolo, la concurrencia y las decisiones de diseño.

9. Criterios de evaluación

Apartado	Peso
Comunicación cliente-servidor	10 %
Protocolo y serialización	15 %
Implementación RPC	25 %
Dispatcher y procedimientos	15 %
Gestión de errores y timeout	15 %
Concurrencia	15 %
Calidad de la documentación	5 %

10. Notas importantes

- ✅ La práctica debe funcionar realmente entre tu ordenador y una instancia EC2.
- ✅ Utiliza solo librerías estándar de Python.
- ✅ No se permite el uso de frameworks RPC (gRPC, RPyC, etc.).
- ✅ Respeta el formato del protocolo definido.
- ✅ En caso de duda, consulta el foro o correo de la asignatura.

> **"La mejor forma de entender RPC es construirlo."**